

Pairing Proofs and Polynomial Ring Toolkit

Shramee Srivastav

[0009-0009-7208-3478]

MIST.cash—FOCBB, Shhtarknet

shramee@proton.me

April 16, 2026

Abstract

Elliptic curve pairings are fundamental to modern zero-knowledge proof systems, including Groth16, PLONK, and KZG polynomial commitments. In resource-constrained execution environments (Ethereum VM, BitVM, blockchain smart contracts) where computation is metered by gas fees, and in arithmetized computation systems (R1CS, Plonkish, Cairo AIR) where each computational step imposes significant prover overhead, the cost of pairing computation remains a practical bottleneck.

We present a public-coin interactive proof system that consolidates the Miller loop iterations and final exponentiation into a single polynomial relation. The relation can then be verified as polynomial identities greatly reducing the pairing costs.

We present a full implementation in gnark [?]¹—the industry-standard framework for arithmetized circuit development—as a reusable polynomial ring toolkit in the `emulated` package. Benchmarked on a Groth16 verification simulation (2 fixed-point pairings + 1 full pairing + 1 $\mathbb{F}_{p^{12}}$ multiplication) on BN254, our gnark implementation achieves a **39% reduction in SCS constraints** (1,890,771 $\rightarrow$ 1,158,194), a **60% reduction in commitments** (739,198 $\rightarrow$ 295,946), and a **~42% reduction** in compile-time memory and up to **52%** in solver memory. These improvements hold across both SCS and R1CS constraint systems and enable practical pairing verification in recursive proof systems and resource-constrained environments.

Contents

List of Algorithms

1	Optimal Ate pairing over Barreto-Naehrig curves [?]	3
2	Optimal Ate pairing with Residue witness check	6
3	PolyMulDiv: Multiply polynomials and divide by P_{12}	11
4	Pairing Prover: Preparing polynomials	12
5	EvalPoly: Evaluate polynomial product minus remainder via Horner’s method	13
6	Pairing Verifier: Evaluating remainders	14

List of Tables

1 Introduction

Blockchain technology continues to mature with improving scalability and infrastructure. However, privacy remains a significant barrier to entry for many users and applications. Zero-knowledge proofs (ZKPs) have emerged as a powerful solution to this challenge.

1.1 Motivation

Elliptic curve pairings represent fundamental building blocks for numerous cryptographic systems including Groth16 [?], PLONK [?], BLS signature schemes [?], and KZG polynomial commitment schemes [?].

However, **pairings are computationally intensive**. This creates critical bottlenecks in two important deployment contexts:

1. **Resource-constrained virtual machines** (Ethereum VM, BitVM, ZKVMs): Computation is metered — each operation incurs gas costs. A single pairing verification requiring thousands of field operations becomes economically prohibitive during peak network congestion.
2. **Arithmetized computation systems** (R1CS [?], Plonkish [?], Cairo AIR [?]): Each operation must be represented algebraically, adding significant prover overhead beyond native execution costs.

In such contexts, an efficient proof system for pairing correctness can replace expensive native execution inside the constrained environment.

1.2 Our Contribution

We present a public-coin interactive proof system for verifying pairing computations that consolidates verification into unified polynomial relationships under the Schwartz-Zippel lemma. By treating complete Miller loop iterations as single polynomial relationships rather than sequences of individual extension field operations [?], our approach achieves:

- **39% reduction** in SCS constraints ($1,890,771 \rightarrow 1,158,194$ for BN254 Groth16 simulation)
- **60% reduction** in commitments ($739,198 \rightarrow 295,946$)
- **~42% reduction** in compile-time memory and up to **52%** in solver memory

Our key insight is that instead of verifying individual field operations separately, we can verify entire Miller loop iterations as single polynomial identities. This consolidation dramatically reduces both the number of modular operations required and the communication overhead from intermediate commitments.

As a reusable artefact of independent interest, we also contribute a **polynomial ring toolkit** for gnark’s emulated arithmetic package. The toolkit packages the two-round interactive proof system into a clean API: a single `NewPolyRingCheck` call registers a modulus, `MulPolyRings` submits multiplications with deferred verification, and

`performDeferredRingChecks` closes the circuit with two `api.Commit` challenges. Atop this, the `PolyRingAccumulator` provides a higher-level interface for building product chains: factors are enqueued via `Mul`, squarings via `Sqr`, and the accumulated product is evaluated lazily through `Eval`, with optional automatic degree-bounding to keep intermediate products manageable.

2 Background

Algorithm 1 Optimal Ate pairing over Barreto-Naehrig curves [?]

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$

Output: $a_{opt}(Q, P)$

```

1: Write  $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$  where  $s_i \in \{-1, 0, 1\}$  and  $s_L = 1$ 
2:  $T \leftarrow Q, f \leftarrow 1$ 
3: for  $i = L - 2$  to  $0$  do
4:    $f \leftarrow f^2 \cdot l_{T,T}(P)$ 
5:    $T \leftarrow [2]T$ 
6:   if  $s_i = 1$  then
7:      $f \leftarrow f \cdot l_{T,Q}(P)$ 
8:      $T \leftarrow T + Q$ 
9:   end if
10:  if  $s_i = -1$  then
11:     $f \leftarrow f \cdot l_{T,-Q}(P)$ 
12:     $T \leftarrow T - Q$ 
13:  end if
14: end for
15:  $Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$ 
16:  $f \leftarrow f \cdot l_{T,Q_1}(P), T \leftarrow T + Q_1$ 
17:  $f \leftarrow f \cdot l_{T,-Q_2}(P), T \leftarrow T - Q_2$ 
18:  $f \leftarrow f^{(p^{12}-1)/r}$ 
19: return  $f$ 

```

2.1 Pairing Computation

An elliptic curve pairing is a non-degenerate bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, where $\mathbb{G}_1$ and $\mathbb{G}_2$ are groups of points on elliptic curves over finite fields, and $\mathbb{G}_T$ is a multiplicative group in an extension field. For practical cryptographic applications, we require that this map be efficiently computable. We will focus on the optimal ate pairing [?] over Barreto-Naehrig (BN) curves, which is widely used due to its efficiency. We will specifically use the Ethereum’s BN254 curve, defined over a 254-bit prime field $\mathbb{F}_q$, with groups $\mathbb{G}_1, \mathbb{G}_2$ of order r and target group $\mathbb{G}_T$ contained in the 12th extension field $\mathbb{F}_{q^{12}}$.

The computation of an optimal ate pairing—the focus of our implementation—consists of two primary phases:

1. **Miller loop:** The core iterative computation that evaluates line functions at pairs of points from $\mathbb{G}_1$ and $\mathbb{G}_2$, accumulating intermediate results to produce a value in $\mathbb{F}_{q^{12}}$.

2. **Final Exponentiation:** This step raises the Miller loop output to the power $(q^{12} - 1)/r$, where r is the order of the groups, ensuring the result lies in the correct subgroup $\mathbb{G}_T$.

While both phases present optimization opportunities, the constraint-based verification of these computations introduces unique challenges and possibilities that differ significantly from native implementations.

2.2 Schwartz-Zippel lemma

The Schwartz-Zippel lemma (or more precisely, the Demillo-Lipton-Schwartz-Zippel lemma [?, ?, ?, ?]) provides a probabilistic method for testing polynomial identities. The lemma states that for a non-zero polynomial $P(x_1, x_2, \dots, x_n)$ of total degree d over a finite field $\mathbb{F}$, when evaluated at uniformly random field elements, the probability that P evaluates to zero is at most $d/|\mathbb{F}|$.

This probabilistic bound becomes negligible for cryptographically sized fields where $d \ll |\mathbb{F}|$, making it practical to verify polynomial identities by evaluation instead of coefficient-wise operations.

2.3 Computation Models

We implement and benchmark our toolkit in gnark [?], the industry-standard Go library for arithmetized circuit development, targeting both SCS (Sparse Constraint System, the native Plonkish backend) and R1CS constraint systems. We map each field multiplication to a constraint (denoted by **C**) to maintain compatibility with the R1CS model used in established literature. In R1CS [?], prover complexity is dominated by multiplication gates while additions are free. To synchronize our theoretical analysis with the benchmarks provided by El Housni [?], we define our cost metric based on field multiplications.

2.3.1 R1CS Cost Model

We measure computational cost in terms of *constraints* (denoted by **C**), representing the fundamental unit of work in Rank-1 constraint systems. One constraint corresponds to one multiplication gate.

Basic Field Operations in $\mathbb{F}_q$:

- **Addition/Subtraction:** Free
- **Multiplication:** One constraint (1C)
- **Division/Inversion:** One multiplication + one equality check

Commitment Overhead: Beyond constraint costs, we account for *calldata* or *commitment overhead*—the number of field elements the prover must commit to (via Fiat-Shamir hashing in non-interactive proofs). For blockchain deployments, this overhead directly impacts transaction costs and is often the dominant factor.

2.3.2 Cost Model Comparison

Operations that are traditionally expensive (inversions, divisions) become cheap in constraint systems when computed via witnesses. This inverts traditional optimization priorities: we can afford inversions to save multiplications, enabling affine coordinate systems and other techniques impractical in native implementations.

Recognizing and exploiting these unique computational properties is essential for unlocking hidden performance potential. Youssef El Housni’s pioneering work **Pairings in Rank-1 Constraint Systems** [?] represents, to our knowledge, the first systematic exploration of these optimization opportunities, providing a breakthrough in the practicality of recursive proving by thoughtfully leveraging the distinctive cost model inherent to constraint-based systems.

3 Related Work

The landscape of efficient pairing verification in constraint systems has evolved through three key contributions that our work directly builds upon.

1. El Housni’s **Pairings in Rank-1 Constraint Systems** [?] established the theoretical framework for optimizing pairings in arithmetized environments, introducing efficient Miller loop formulas in affine coordinates and sparse line function representations. This work inverts the usual cost model: inversions become cheap (2C) while multiplications remain the dominant cost.
2. Feltroidprime [?] introduced polynomial identity testing for $\mathbb{F}_{q^{12}}$ arithmetic via the Schwartz-Zippel lemma. Each multiplication $A \cdot B$ is expressed as $A(x) \cdot B(x) = Q(x) \cdot P_{12}(x) + R(x)$ and verified at a single random point, with the quotients Q batched via random linear combination. Our work applies the same polynomial framework but at a coarser granularity: entire Miller loop steps rather than individual multiplications, eliminating intermediate remainder commitments entirely.
3. Novakovic and Eagen’s **On Proving Pairings** [?] showed that final exponentiation can be replaced by a residue witness check, reducing final exponentiation cost by over 85% for BN254. We integrate their residue witness c directly into our polynomial step equations (Section ??), unifying the Miller loop and final exponentiation into the same polynomial verification framework.

4 Pairing Operations as Polynomials

Algorithm ?? gives the pairing computation combining El Housni’s R1CS optimizations [?] with Novakovic-Eagen’s residue witness technique [?]. The 27th root of unity W scales the Miller loop output f to be a cubic residue (see [?] §4.3); if f is not already a cubic residue, multiplying by W^w , $w \in \{1, 2\}$ corrects this.

Our contribution is to express each step of this algorithm as a single multi-operand polynomial product verified via the Schwartz-Zippel lemma, rather than as a sequence of individual binary multiplications as in [?].

Algorithm 2 Optimal Ate pairing with Residue witness check

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0, 1, 2\}$

Internal $W, W^2 \in \mathbb{F}_{q^{12}}$

▷ W is 27th root of unity

Output: $a_{opt}(Q, P)$

1: Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$

2: $c^{-1} \leftarrow 1/c$

▷ Inverse Residue witness

3: $f \leftarrow c^{-1}$

▷ Initialize with inverse residue witness

4: $T \leftarrow Q$

▷ T_i for each pair for multi-pairing

Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

5: **for** $i = L - 2$ to 0 **do**

6: $f \leftarrow f^2$

7: $f \leftarrow f \cdot l_{T,T}(P)$

▷ Repeat for T_i, P_i

8: $T \leftarrow [2]T$

▷ Repeat for T_i

9: **if** $s_i = 1$ **then**

10: $f \leftarrow f \cdot c^{-1}$

▷ Residue witness

11: $f \leftarrow f \cdot l_{T,Q}(P)$

▷ Repeat for T_i, Q_i, P_i

12: $T \leftarrow T + Q$

▷ Repeat for T_i, Q_i

13: **else if** $s_i = -1$ **then**

14: $f \leftarrow f \cdot c$

▷ Residue witness

15: $f \leftarrow f \cdot l_{T,-Q}(P)$

▷ Repeat for $T_i, -Q_i, P_i$

16: $T \leftarrow T - Q$

▷ Repeat for $T_i, -Q_i$

17: **end if**

18: **end for**

19: $Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$

▷ Repeat for Q_{1i}, T_i

20: $f \leftarrow f \cdot l_{T,Q_1}(P)$

▷ Repeat for Q_{1i}, T_i, P_i

21: $T \leftarrow T + Q_1$

▷ Repeat for Q_{1i}, T_i

22: $f \leftarrow f \cdot l_{T,-Q_2(Q)}(P)$

▷ Repeat for Q_i, T_i, P_i

▷ Skip $T \leftarrow T - Q_2(Q)$

23: **if** $w = 1$ **then**

24: $f \leftarrow f \cdot W$

25: **else if** $w = 2$ **then**

26: $f \leftarrow f \cdot W^2$

27: **end if**

28: $f \leftarrow f \cdot c^{-q} \cdot c^{q^2} \cdot c^{-q^3}$

▷ Uses Frobenius maps, Negative exponents use c^{-1}

29: **return** f

4.1 Comparative Analysis

We compare the two schemes for a 3-pair Miller loop zero-bit step (the most common case in BN254’s NAF [?] representation of $s = 6t + 2$). Here $F \in \mathbb{F}_{q^{12}}$ is the accumulator, $L_i \in \text{Sparse}_{01379}$ are the three doubling line functions, $R \in \mathbb{F}_{q^{12}}$ is the updated accumulator, and Q is the quotient polynomial. Sparse_{01379} denotes the subspace of $\mathbb{F}_{q^{12}}$ elements whose only nonzero coefficients are at degrees 0, 1, 3, 7, 9—the form that BN254 line functions naturally take [?], reducing their evaluation cost from 12C to 4C.

4.1.1 Previous scheme

Adapting [?] to lines ??–?? of Algorithm ?? for a 3-pair Miller loop gives:

$$\begin{aligned} F(x) \cdot F(x) &\stackrel{?}{=} Q_1(x) \cdot P_{12}(x) + T_1(x) \\ T_1(x) \cdot L_1(x) &\stackrel{?}{=} Q_2(x) \cdot P_{12}(x) + T_2(x) \\ T_2(x) \cdot L_2(x) &\stackrel{?}{=} Q_3(x) \cdot P_{12}(x) + T_3(x) \\ T_3(x) \cdot L_3(x) &\stackrel{?}{=} Q(x) \cdot P_{12}(x) + R(x) \end{aligned}$$

Evaluating F , T_1 – T_3 , R costs 12C each and L_1 – L_3 cost 4C each; all four remainder polynomials require commitment. **Total: 76C and 48 $\mathbb{F}_q$ commitments** (4×12).

4.1.2 Our scheme

Our scheme consolidates lines ??–?? into a single polynomial identity:

$$F(x) \cdot F(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) \stackrel{?}{=} Q(x) \cdot P_{12}(x) + R(x)$$

By eliminating intermediate T_* polynomials, only R requires commitment. **Total: 41C and 12 $\mathbb{F}_q$ commitments** ($2 \times 12C + 3 \times 4C + 5C$).

This results in **46% fewer constraints** and a **75% reduction in commitments**. When verifying ZK proofs on-chain, commitment savings are often more critical than constraint savings, due to the calldata costs associated with transmitting polynomial hints.

4.1.3 Performance Comparison

Table ?? compares the computational costs and commitment overhead between the schemes for a single 0-bit Miller loop operation with 3 pairs.

4.2 Operation Polynomials

Our implementation uses different polynomials for different bits in the Miller loop. Let us walk through a 3-pairing setup which Inversions used widely for Groth16 verification. We will describe the polynomials for zero bits, non-zero bits and correction step.

Table 1: 3-Pair Miller loop 0-bit constraints comparison

Scheme	Constraints	$\mathbb{F}_q$ Commitments	Improvement
[?]	132C	—	Baseline
[?]	76C*	48	42%
Our scheme	41C*	12	69%
[?]: 1 square (36C) + 3 sparse mul (3×30C) = 132C [?]: 5 evals (5×12C) + 3 line evals (3×4C) + 4 muls (4×1C) = 76C Our scheme: 2 evals (2×12C) + 3 line evals (3×4C) + 5C (muls + RLC) = 41C *Costs exclude Fiat-Shamir commitment overhead			

4.2.1 Miller loop: Zero Bit Equation

As seen in ?? for zero bit operations (line ?? and ?? with three $l_{T,T}(P)$ lines in Algorithm ??), the verification equation is:

$$F^2(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) = R(x) + Q(x) \cdot P_{12}(x) \quad (1)$$

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	$11 \times 2 = 22$	12
$L_1, L_2, L_3 \in \text{Sparse}_{01379}$	The line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	36C Evals, 4C Muls and 1C RLC		41
Q (<i>Amortized See ??</i>)	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	38	39

Table 2: Zero bit operation polynomial components

4.2.2 Miller loop: Non-Zero Bit Equation

For non-zero bit operations (lines ??, ??, ??, ?? with three pairs of lines in Algorithm ??), the Miller loop performs both doubling and addition steps. So for each pair we have two Sparse_{01379} lines. The verification equation becomes:

$$F^2(x) \cdot W(x) \cdot L_{d1}(x) \cdot L_{d2}(x) \cdot L_{d3}(x) \cdot L_{a1}(x) \cdot L_{a2}(x) \cdot L_{a3}(x) = R(x) + Q(x) \cdot P_{12}(x) \quad (2)$$

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	$11 \times 2 = 22$	12
$L_{d1}, L_{d2}, L_{d3} \in \text{Sparse}_{01379}$	Doubling line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
$L_{a1}, L_{a2}, L_{a3} \in \text{Sparse}_{01379}$	Addition line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
W or $W^{-1} \in \mathbb{F}_{q^{12}}$	Residue witness	11	12
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	60C Evals, 8C Muls and 1C RLC		69
Q (<i>Amortized See ??</i>)	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	76	77

Table 3: Non-zero bit operation polynomial components

The residue witness W encapsulates the equivalence class relationship established by Novakovic and Eagen’s final exponentiation elimination. The inverse of W is used for negative bits, this just changes the coefficients the equation remains the same.

4.2.3 Miller loop: Frobenius mapping Equation

The correction step finalizes the Miller loop by incorporating the Frobenius endomorphism corrections (lines ?? and ?? in Algorithm ??). Again, we have two Sparse_{01379} lines for each pair, for π_p and $-\pi_p^2$ mappings, without any squaring of the accumulator. The verification equation becomes:

$$F(x) \cdot L_{1\pi}(x) \cdot L_{2\pi}(x) \cdot L_{3\pi}(x) \cdot L_{1\pi^2}(x) \cdot L_{2\pi^2}(x) \cdot L_{3\pi^2}(x) = R(x) + Q(x) \cdot P_{12}(x) \quad (3)$$

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	11	12
$L_{1\pi}, L_{2\pi}, L_{3\pi} \in \text{Sparse}_{01379}$	π_p correction lines	$9 \times 3 = 27$	$4 \times 3 = 12$
$L_{1\pi^2}, L_{2\pi^2}, L_{3\pi^2} \in \text{Sparse}_{01379}$	$-\pi_p^2$ correction lines	$9 \times 3 = 27$	$4 \times 3 = 12$
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	48C Evals, 6C Muls and 1C RLC		55
Q (<i>Amortized See ??</i>)	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	54	55

Table 4: Correction step polynomial components

4.2.4 Final Exponentiation Equation

The final exponentiation check is handled as described in [?]. The equation handles the cubic scale factor multiplication, and **Frobenius component** ($q - q^2 + q^3$) of the final exponentiation (lines ??, ?? and ?? in Algorithm ??). The verification equation looks like:

$$F(x) \cdot c^{-q}(x) \cdot c^{q^2}(x) \cdot c^{-q^3}(x) \cdot W^w(x) = 1 + Q(x) \cdot P_{12}(x) \quad (4)$$

where $w \in \{0, 1, 2\}$ determines the cubic root of unity scaling factor, and the Frobenius mappings are computed using the residue witness c and its inverse and provided as input. Cubic root is sparse element with just 2 coefficients the degree ranging from 8 to 10 because of positioning of the coefficients. Our R is now 1 for a successful proof verification.

With this we have all the polynomials we need for verifying entire pairing operation. Next we will look into high level overview of pairing verification via an interactive oracle proof.

4.3 Security

Each verification equation takes the form $P = \prod_i A_i(x) - Q(x) \cdot P_{12}(x) - R(x)$ where $\deg(P) < 2^8$.¹ By the Schwartz-Zippel lemma (Section ??):

¹For k -pair pairings, $\deg(P) \approx 88 + 18(k - 3)$, well below 2^8 for all practical k . This bound can be extended to 2^{10} with negligible soundness impact.

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Final Miller loop result	11	12
$c^{-q} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius $-q$	11	12
$c^{q^2} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius q^2	11	12
$c^{-q^3} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius $-q^3$	11	12
$W^w \in \mathbb{F}_{q^{12}}$	Cubic scale factor	10	2
$R = 1$	Unity (constant)	0	1
Total constraints	51C Evals, 4C Muls and 1C RLC		56
Q (<i>Amortized See ??</i>)	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	43	44

Table 5: Final exponentiation polynomial components

Soundness: $\delta_s \leq d/|\mathbb{F}_q| < 2^8/2^{254} = 2^{-246}$ for BN254.

Completeness: An honest prover always produces $P(x) = 0$ identically, so the verifier always accepts. Euclidean division guarantees unique Q, R with $\deg(R) < \deg(P_{12}) = 12$.

5 Full Pairing Proof with Quotient Batching

We now combine the polynomial equations from Section ?? into a complete two-round public-coin interactive proof, further reducing verification cost by batching all quotients into a single accumulated polynomial.

5.1 Batching Polynomial Identity Testing

All n polynomial equations from Section ?? are batched into a single verification. The prover commits to remainders $\{R_i\}_{i=0}^{n-1}$ and receives challenge z . Using z , the prover forms the accumulated quotient $Q_{\text{acc}} = \sum_{i=0}^{n-1} z^i \cdot Q_i$. The verifier then samples a second challenge point c and checks:

$$\sum_{i=0}^{n-1} z^i \cdot [A_i(c) \cdot B_i(c) \cdots X_i(c) - R_i(c)] = Q_{\text{acc}}(c) \cdot P_{12}(c) \quad (5)$$

This reduces n polynomial verifications (degrees 38–76, see Tables ??–??) to a single evaluation of a degree-76 polynomial.

5.2 Prover's Algorithms

The prover computes quotients Q_i and remainders R_i for each equation in Section ?. For each step, it calls POLYMULTIDIV (Algorithm ??), which multiplies the input polynomials and performs standard Euclidean division by the irreducible P_{12} . After receiving challenge z , the prover forms $Q_{\text{acc}} = \sum_i z^i Q_i$ and sends $\mathcal{R} = \{R_i\}$ and Q_{acc} to the verifier. The full prover procedure is given in Algorithm ??

5.3 Verifier's Algorithms

The verifier receives $P \in \mathbb{G}_1$, $Q \in \mathbb{G}_2$, residue witness c [?], the W (27th root of unity), the remainder array $\mathcal{R}$, and Q_{acc} from the prover. Using challenge z from the first round and

Algorithm 3 PolyMulDiv: Multiply polynomials and divide by P_{12}

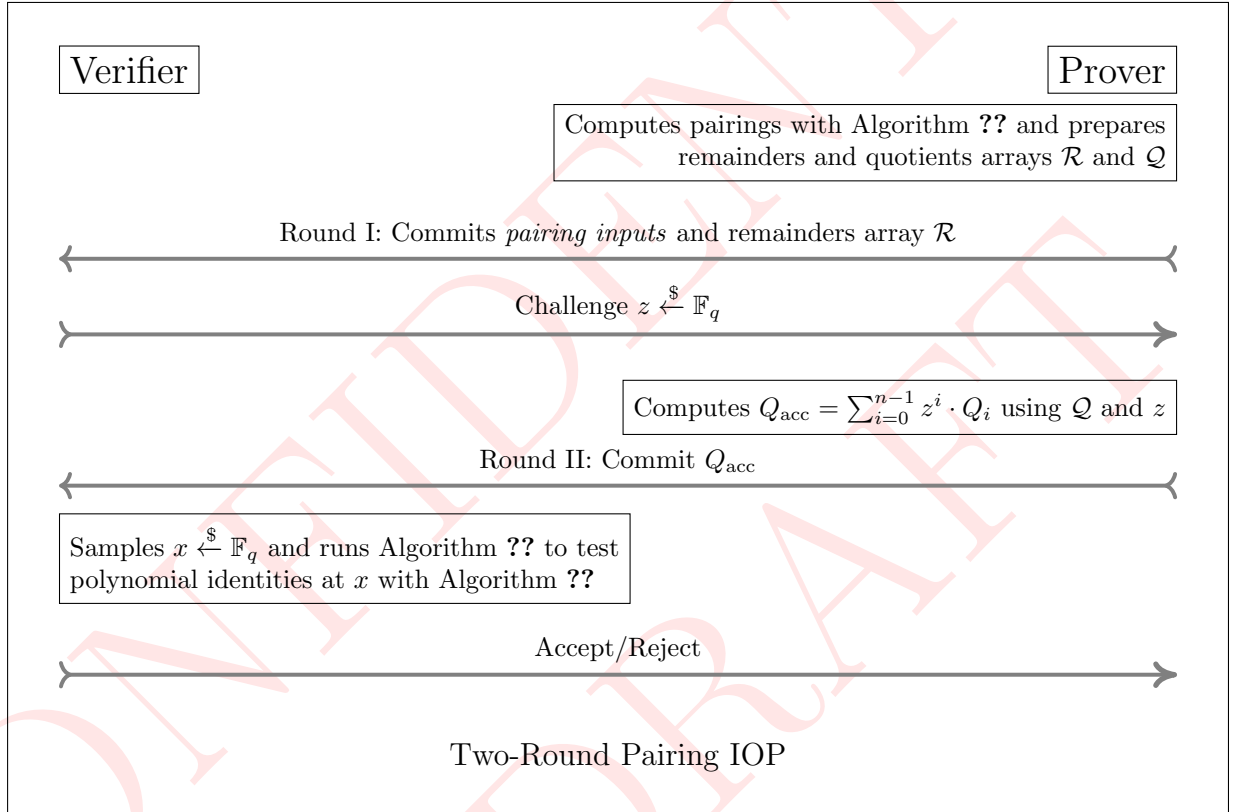
Input: $P_1, \dots, P_n \in \mathbb{F}_q[x]$; Divisor $P_{12} \in \mathbb{F}_q[x]$

Output: Q, R s.t. $\prod P_i = Q \cdot P_{12} + R$, $\deg(R) < \deg(P_{12})$

- 1: $R \leftarrow \prod_{i=1}^n P_i$ $\triangleright$ Polynomial product
 - 2: Perform Euclidean division of R by P_{12} to get (Q, R)
 - 3: **return** (Q, R)
-

a freshly sampled x , it mirrors the prover's pairing loop while evaluating each polynomial at x via Algorithm ??, accumulating the RLC e_{acc} . The final check is Equation ??. The full verifier is given in Algorithm ??.

5.4 Interactive Proof



5.5 Fiat-Shamir Transformation

Applying the Fiat-Shamir transformation [?] with random oracle H gives a non-interactive proof with knowledge soundness $\delta_s + 3(m^2 + 1)2^{-\lambda}$ [?], where $m = 2$ (hash queries) and $\lambda = 254$ (BN254 hash output length).

Gnark's multi-commitment mechanism. In gnark, each `api.Commit` call registers committed variables in the constraint system; the two sequential calls in our protocol produce the two challenges z and x via gnark's multi-round commitment scheme [?, ?].

Algorithm 4 Pairing Prover: Preparing polynomials

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0, 1, 2\}$ Internal: $W, W^2 \in \mathbb{F}_{q^{12}}$ $\triangleright W$ is 27th root of unity**Output:** Arrays $\mathcal{Q}, \mathcal{R}$ of quotients and remaindersWrite $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$ 1: $c^{-1} \leftarrow 1/c$ $\triangleright$ Inverse Residue witness2: $f \leftarrow c^{-1}$ $\triangleright$ Initialize with inverse residue witness3: $T \leftarrow Q$ $\triangleright T_i$ for each pair for multi-pairingEvery operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

Miller loop bits

4: **for** $i = L - 2$ to 0 **do**5: $L_{\text{dbl}} \leftarrow l_{T,T}(P), T \leftarrow [2]T$ 6: **if** $s_i = 0$ **then**7: $Q, R \leftarrow \text{PolyMulDiv}(f, f, L_{\text{dbl}})$ $\triangleright$ Algorithm ??, add L_{*i} for each P_i, Q_i 8: **else if** $s_i = 1$ **then**9: $L_{\text{add}} \leftarrow l_{T,Q}(P), T \leftarrow T + Q$ 10: $Q, R \leftarrow \text{PolyMulDiv}(f, f, c^{-1}, L_{\text{dbl}}, L_{\text{add}})$ $\triangleright$ Algorithm ??, add L_{*i} for each P_i, Q_i 11: **else if** $s_i = -1$ **then**12: $L_{\text{add}} \leftarrow l_{T,-Q}(P), T \leftarrow T - Q$ 13: $Q, R \leftarrow \text{PolyMulDiv}(f, f, c, L_{\text{dbl}}, L_{\text{add}})$ $\triangleright$ Algorithm ??, add L_{*i} for each P_i, Q_i 14: **end if**15: $f \leftarrow R, \mathcal{Q}.\text{append}(Q), \mathcal{R}.\text{append}(R)$ 16: **end for**

Miller loop Frobenius correction

17: $Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$ 18: $L_\pi \leftarrow l_{T,Q_1}(P)$ 19: $T \leftarrow T + Q_1$ 20: $L_{-\pi^2} \leftarrow l_{T,-Q_2}(P)$ 21: $Q, R \leftarrow \text{PolyMulDiv}(f, L_\pi, L_{-\pi^2})$ $\triangleright$ Algorithm ??, add L_{*i} for each P_i, Q_i 22: $f \leftarrow R, \mathcal{Q}.\text{append}(Q), \mathcal{R}.\text{append}(R)$

Final exponentiation: Cubic scaling + Frobenius mappings

23: **if** $w = 0$ **then**24: $Q, R \leftarrow \text{PolyMulDiv}(f, c^{-q}, c^{q^2}, c^{-q^3})$ $\triangleright$ Algorithm ??25: **else if** $w = 1$ **then**26: $Q, R \leftarrow \text{PolyMulDiv}(f, W, c^{-q}, c^{q^2}, c^{-q^3})$ $\triangleright$ Algorithm ??27: **else if** $w = 2$ **then**28: $Q, R \leftarrow \text{PolyMulDiv}(f, W^2, c^{-q}, c^{q^2}, c^{-q^3})$ $\triangleright$ Algorithm ??29: **end if**30: $\mathcal{Q}.\text{append}(Q), \mathcal{R}.\text{append}(R)$ $\triangleright$ Final $\mathcal{Q}$ and $\mathcal{R}$ accumulation, skip f 31: **return** $\mathcal{Q}, \mathcal{R}$

Algorithm 5 EvalPoly: Evaluate polynomial product minus remainder via Horner's method

Input: Polynomials $P_1, \dots, P_n, R \in \mathbb{F}_q[x]$ of degree $\leq d$, evaluation point $z \in \mathbb{F}_q$

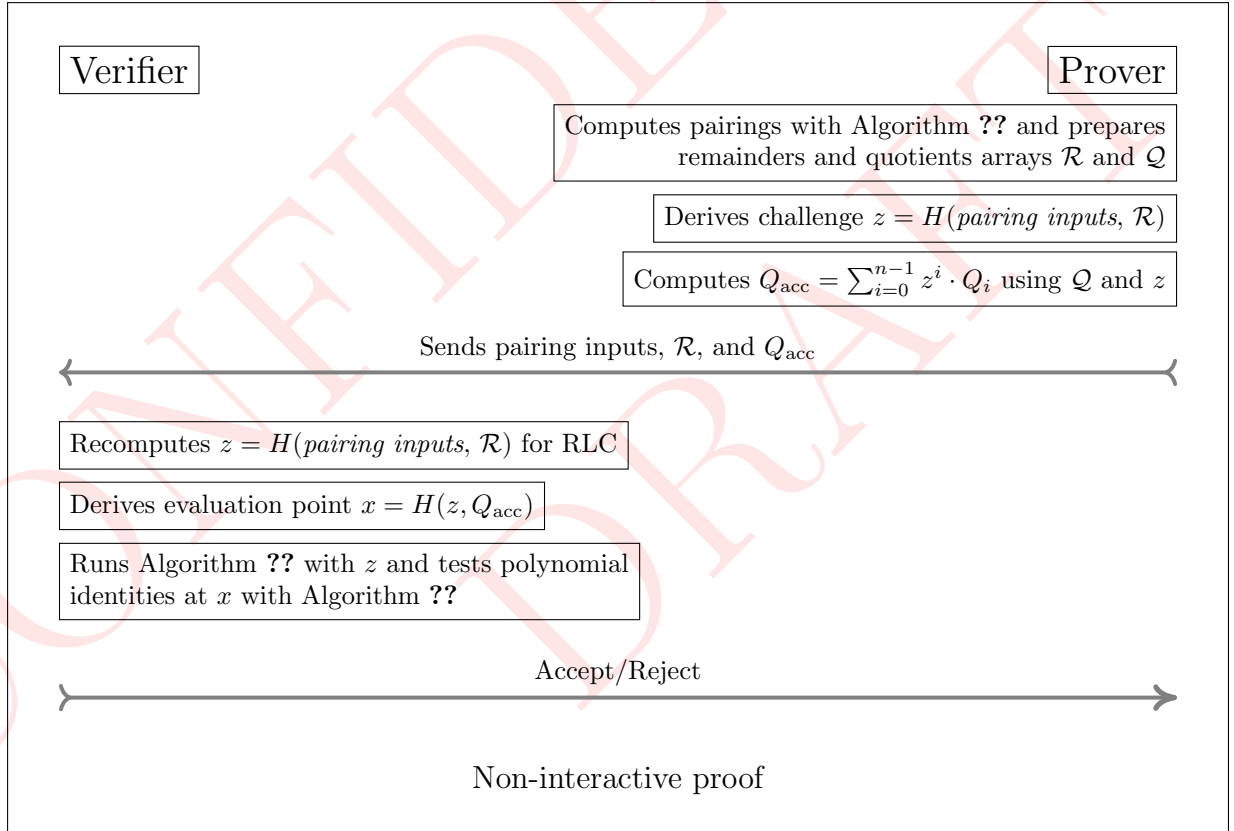
Output: $e \in \mathbb{F}_q$ where $e = \prod_{i=1}^n P_i(z) - R(z)$

```

1: function EVAL( $P = (p_0, p_1, \dots, p_d), z$ )                                ▷ Uses Horner's method
2:    $v \leftarrow p_d$                                                          ▷ Start from leading coefficient
3:   for  $k = d - 1$  downto 0 do
4:      $v \leftarrow v \cdot z + p_k$                                            ▷  $d$  multiplications,  $d$  additions total
5:   end for
6:   return  $v$ 
7: end function
8:  $e \leftarrow \text{EVAL}(P_1, z)$                                                ▷ Evaluate first polynomial
9: for  $i = 2$  to  $n$  do
10:   $e \leftarrow e \cdot \text{EVAL}(P_i, z)$                                      ▷ Multiply in each subsequent evaluation
11: end for
12:  $e \leftarrow e - \text{EVAL}(R, z)$                                            ▷ Subtract remainder evaluation
13: return  $e$ 

```

5.6 Non-Interactive Proof



Concrete commitment costs: In gnark, each call to `api.Commit` registers committed variables in the constraint system; the precise prover overhead depends on the backend (Groth16 or Plonk) [?, ?].

Algorithm 6 Pairing Verifier: Evaluating remainders

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0, 1, 2\}, \mathcal{R}$ array of remainders, $Q_{\text{acc}} \in \mathbb{F}_q[x]$

Internal: 27th root of unity, $W, W^2 \in \mathbb{F}_{q^{12}}$, first round challenge, $z \xleftarrow{\$} \mathbb{F}_q$

Output: $\text{pairing}(P, Q) \stackrel{?}{=} 1$

Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$

- 1: $c^{-1} \leftarrow 1/c$ ▷ Inverse Residue witness
 - 2: $f \leftarrow c^{-1}$ ▷ Initialize with inverse residue witness
 - 3: $T \leftarrow Q$ ▷ T_i for each pair for multi-pairing
 - 4: $x \xleftarrow{\$} \mathbb{F}_q$ ▷ Sample evaluation point from field
 - 5: $e_{\text{acc}} \leftarrow 0, a_{\text{rlc}} \leftarrow 1$ ▷ Evaluation accumulator and random linear combination multiplier
- Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

Miller loop bits

- 6: **for** $i = L - 2$ to 0 **do**
- 7: $L_{\text{dbl}} \leftarrow l_{T,T}(P), T \leftarrow [2]T$
- 8: $R \leftarrow \mathcal{R}.\text{pop_front}()$ ▷ Fetch R from prover
- 9: **if** $s_i = 0$ **then**
- 10: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, f, L_{\text{dbl}})$ ▷ Algo ??, add L_{*i} for each P_i, Q_i
- 11: **else if** $s_i = 1$ **then**
- 12: $L_{\text{add}} \leftarrow l_{T,Q}(P), T \leftarrow T + Q$
- 13: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, f, c^{-1}, L_{\text{dbl}}, L_{\text{add}})$ ▷ Algo ??, add L_{*i} for each P_i, Q_i
- 14: **else if** $s_i = -1$ **then**
- 15: $L_{\text{add}} \leftarrow l_{T,-Q}(P), T \leftarrow T - Q$
- 16: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, f, c, L_{\text{dbl}}, L_{\text{add}})$ ▷ Algo ??, add L_{*i} for each P_i, Q_i
- 17: **end if**
- 18: $f \leftarrow R, a_{\text{rlc}} \leftarrow a_{\text{rlc}} \cdot z$ ▷ Update random linear combination factor
- 19: **end for**

Miller loop Frobenius correction

- 20: $R \leftarrow \mathcal{R}.\text{pop_front}()$ ▷ Fetch R from prover
- 21: $Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$
- 22: $L_{\pi} \leftarrow l_{T,Q_1}(P)$
- 23: $T \leftarrow T + Q_1$
- 24: $L_{-\pi^2} \leftarrow l_{T,-Q_2}(P)$
- 25: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, L_{\pi}, L_{-\pi^2})$ ▷ Algo ??, add L_{*i} for each P_i, Q_i
- 26: $f \leftarrow R, a_{\text{rlc}} \leftarrow a_{\text{rlc}} \cdot z$ ▷ Update random linear combination factor

Final exponentiation: Cubic scaling + Frobenius mappings

- 27: $R \leftarrow \mathcal{R}.\text{pop_front}()$ ▷ Fetch R from prover
- 28: **if** $w = 0$ **then**
- 29: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algo ??
- 30: **else if** $w = 1$ **then**
- 31: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, W, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algo ??
- 32: **else if** $w = 2$ **then**
- 33: $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * \text{EvalPoly}(R, x, f, W^2, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algo ??
- 34: **end if**

- 35: **return** $Q_{\text{acc}}(x) \stackrel{?}{=} e_{\text{acc}} \wedge R \stackrel{?}{=} 1$ ▷ Polynomial identity and expected pairing output check
-

5.7 Security Analysis

Throughout this section, $z, x \in \mathbb{F}_q$ denote field element challenges (scalars), while polynomials are elements of $\mathbb{F}_q[x]$ with x as the formal indeterminate. In particular, z^j is a scalar coefficient, not a polynomial term.

5.7.1 Algebraic Intuition

Before the formal soundness proof it is instructive to consider why an incorrect remainder is difficult to hide. Suppose a malicious prover corrupts exactly one remainder: $R'_j = R_j + \Delta_j$ with $\Delta_j \in \mathbb{F}_q[x]$, $\Delta_j \neq 0$, $\deg(\Delta_j) \leq 11$. Since z is obtained *after* the commitment to $\mathcal{R}$, the prover cannot adjust any other R_k to compensate for Δ_j . To successfully deceive the verifier, the prover must find a valid forged quotient $Q'_{\text{acc}} \in \mathbb{F}_q[x]$ satisfying

$$\sum_{i=0}^{n-1} z^i (A_i - R'_i) = Q'_{\text{acc}} \cdot P_{12}$$

All $R'_i = R_i$ except for $R'_j = R_j + \Delta_j$. Substituting $R'_j = R_j + \Delta_j$ gives

$$\begin{aligned} Q'_{\text{acc}} \cdot P_{12} &= Q_{\text{acc}} \cdot P_{12} - z^j \Delta_j(x) \\ Q'_{\text{acc}} &= Q_{\text{acc}} - \frac{z^j \Delta_j(x)}{P_{12}(x)}. \end{aligned}$$

Here $z^j \in \mathbb{F}_q$ is a nonzero scalar (with overwhelming probability over the choice of z) and $\Delta_j(x) \in \mathbb{F}_q[x]$ is a nonzero polynomial of degree at most 11. Since P_{12} is irreducible of degree 12 and $\deg(z^j \Delta_j) \leq 11 < 12 = \deg(P_{12})$, the polynomial P_{12} does not divide $z^j \Delta_j$ in $\mathbb{F}_q[x]$; hence $z^j \Delta_j / P_{12}$ is not a polynomial. Consequently no valid $Q'_{\text{acc}} \in \mathbb{F}_q[x]$ can satisfy the equation for this single-corruption case.

The same degree argument extends immediately to an adversary who corrupts *multiple* remainders: for any scalar z , the weighted sum $\sum_i z^i \Delta_i(x)$ is still a polynomial in x of degree at most 11 (since each z^i is a scalar and each $\deg(\Delta_i) \leq 11$), so it cannot be divisible by P_{12} of degree 12 unless it is the zero polynomial, which requires every $\Delta_i = 0$. The formal soundness proof below re-derives this uniformly via Schwartz-Zippel.

5.7.2 Soundness

[Soundness] Let n be the number of polynomial equations batched via a random linear combination into the bivariate polynomial identity:

$$P_{\text{final}}(z, x) = \sum_{i=0}^{n-1} z^i \cdot \left[\prod_j P_{i,j}(x) - R_i(x) \right] - Q_{\text{acc}}(x) \cdot P_{12}(x) = 0$$

Let the maximum degree in x of the polynomial products be $d_x < 2^8$. For a prover committing to incorrect remainders, the probability of successful forgery δ_s over randomly chosen $(z, x) \in \mathbb{F}_q^2$ is bounded by:

$$\delta_s \leq \frac{n - 1 + d_x}{|\mathbb{F}_q|}$$

Proof. If the prover commits to an incorrect remainder $R_i(x)$, the evaluation yields a non-zero error polynomial $P_{\text{err}}(z, x)$. The structure of $P_{\text{final}}(z, x)$ establishes the following degree bounds for $P_{\text{err}}(z, x)$:

- **Degree in z :** at most $n - 1$ (due to the z^i weighting).
- **Degree in x :** at most $d_x < 2^8$ (due to the polynomial products).

By the Schwartz-Zippel lemma, the probability that the non-zero polynomial P_{err} evaluates to zero is bounded by the total degree divided by the field size:

$$\Pr[P_{\text{err}}(z, x) = 0] \leq \frac{\deg_z(P_{\text{err}}) + \deg_x(P_{\text{err}})}{|\mathbb{F}_q|} \leq \frac{n - 1 + d_x}{|\mathbb{F}_q|}$$

□

Concrete parameters for BN254: With $n = 67$ equations and $d_x \leq 256$:

$$\delta_s \leq \frac{66 + 256}{2^{254}} < \frac{2^9}{2^{254}} = 2^{-245}.$$

Non-interactive proof via Fiat-Shamir: Under the random oracle model the knowledge soundness error becomes [?]:

$$\delta'_s = \delta_s + 3(m^2 + 1) \cdot 2^{-\lambda}$$

where $m = 2$ (hash queries) and $\lambda = 254$ (hash output length), giving

$$\delta'_s \leq 2^{-245} + 15 \cdot 2^{-254} < 2^{-244}.$$

[Correctness of polynomial evaluation] Theorem ?? assumes that the verifier evaluates all polynomials at the challenge point x without arithmetic error. The circuit implementation uses the deferred-reduction inner-product optimisation of Section ?? (Section ??), which accumulates limb-wise products over $\mathbb{Z}$ before a single final reduction. Correctness of this optimisation requires that no intermediate limb value reaches the native field characteristic p . We verify this bound concretely for BN254 in Lemma ?? (Section ??).

5.7.3 Completeness

For an honest prover executing Algorithm ?? with valid inputs:

1. **Polynomial construction:** At each operation, the prover computes following polynomials:

$$\prod_j P_{i,j} = Q_i \cdot P_{12} + R_i$$

2. **RLC formation:** Given challenge z , the prover forms:

$$Q_{\text{acc}} = \sum_{i=0}^{n-1} z^i \cdot Q_i$$

3. **Verifier's check:** The verifier evaluates at point x :

$$\begin{aligned}
\text{LHS} &= \sum_{i=0}^{n-1} z^i \cdot \left[\prod_j P_{i,j}(x) - R_i(x) \right] \\
&= \sum_{i=0}^{n-1} z^i \cdot Q_i(x) \cdot P_{12}(x) \quad (\text{by construction}) \\
&= P_{12}(x) \cdot \sum_{i=0}^{n-1} z^i \cdot Q_i(x) \\
&= P_{12}(x) \cdot Q_{\text{acc}}(x) = \text{RHS}
\end{aligned}$$

Since the equality holds for any evaluation point x , the protocol satisfies perfect completeness.

6 Polynomial Ring Toolkit

The protocol of Section ?? is packaged as a reusable toolkit in the `emulated` package of gnark [?], parameterised by any emulated field $\mathbb{F}_q$ (via `FieldParams` type T) and any modulus polynomial $P \in \mathbb{F}_q[x]$. The toolkit operates in a recursive proof setting where the prover's computations are performed outside the circuit via `hints` (Section ??). And verifier's checks are implemented as constraints inside the circuit.

6.1 Ring Representation

A ring element is essentially a coefficient slice `[]*Element[T]` in ascending degree order. A `nil` entry is treated as zero by `InnerProductNoReduce`—the term is skipped entirely—so sparse polynomials carry no wasted constraint variables.

The optional `modEvalFn` of type `EvalFnType[T] = ([*Element[T]]) → *Element[T]` allows handling small-integer coefficients efficiently by multiplying by the small positive integer first and negating afterwards, rather than multiplying by the full-width field representative $q - c$.

Example: BN254 $\mathbb{F}_{p^{12}}$

$P_{12}(x) = x^{12} - 18x^6 + 82$ has nine zero coefficients (stored as `nil`) and two small non-zero ones. The `modEvalFn` computes $P_{12}(\alpha) = \text{MulConst}(\alpha^0, 82) + \text{Neg}(\text{MulConst}(\alpha^6, 18)) + \alpha^{12}$, keeping intermediate values small instead of multiplying by $q - 18$.

6.2 Hint Functions

Hints allow defining computations outside of a circuit. A hint defines an annotated function in the circuit at compile time. Inputs and outputs are injected at the solving time. We define these two hints to implement the prover's steps in the two-round protocol of Section ??.

`polyRingMulHint` implements `POLYMULDIV` (Algorithm ??): it multiplies the input polynomials, divides by P to obtain (Q, R) , and serialises the result as `nbQTerms` | Q limbs | `nbRemTerms` | R limbs. Inputs are packed as `nbBits` | `nbLimbs` | `nbPoly` | q | inputs | P , where each polynomial is prefixed by its term count.

`quotientsRLCHint` computes the accumulated quotient $Q_{\text{acc}}[j] = \sum_i z^i \cdot Q_i[j] \bmod q$ coefficient-wise, corresponding to the prover's step in Algorithm ??.

6.3 API

Register a ring group

```
group := f.NewPolyRingCheck(mod, modEvalFn)
```

Creates a `PolyRingGroupChecks[T]`, appends it to the field's deferred check list, and returns a handle shared by all multiplications under the same modulus. Pass `nil` for `modEvalFn` to use the dense fallback. Multiple groups with different moduli can coexist in one circuit.

Submit a multiplication

```
rem, err := f.MulPolyRings(group, A, B, C, ...)
```

Computes $R = (\prod_j A_j) \bmod P$ via `polyRingMulHint` outside the circuit, range-checks R 's limbs inside (`packLimbs` with `strict=true`), and registers the check $(A, B, C; R; Q)$ in `group`. The quotient Q is not range-checked; it is consumed only by the RLC hint outside the circuit. The returned `rem` is a fully reduced `*Poly[T]` and can be fed directly into the next `MulPolyRings` call.

Accumulate a product chain

```
acc := f.NewPolyRingAccumulator(group, targetDeg)
acc.Mul(A)
acc.Mul(B)
acc.Sqr() // enqueues (A·B)2 without multiplying yet
rem := acc.Eval()
```

`PolyRingAccumulator` maintains a queue of polynomial factors representing the deferred product $\prod_i \text{state}_i$. Calling `Mul` appends a factor; calling `Sqr` duplicates every current factor so that the next `Eval` computes $(\prod_i \text{state}_i)^2$. When `targetDeg` is set, `Eval` is triggered automatically whenever the accumulated degree would exceed the bound, keeping intermediate products within a manageable degree before the final `MulPolyRings` call. Without a target degree all operations are deferred until `Eval` is called explicitly.

Deferred verification

```
f.performDeferredRingChecks(api) // called once at circuit finalisation
```

Implements Equation (??) via two sequential `api.Commit` calls [?] (producing challenges z and x) and one `AssertIsEqual` per group. Gnark derives $z = \text{MiMC}(\text{root})$ from the first call and $x = \text{MiMC}(z)$ from the second, chaining the challenges across rounds (see Section ??). No in-circuit work beyond remainder range-checks is performed until this point.

6.4 Usage in Practice

Register the ring group once at initialisation:

```
fp, _ := emulated.NewField[emulated.BN254Fp](api)
modPoly := fp.MakePoly([]any{82, 0, 0, 0, 0, 0, -18, 0, 0, 0, 0, 0, 1})
ring := fp.NewPolyRingCheck(modPoly, nil)
```

`MakePoly` accepts Go integer literals and maps zeros to `nil`, so the slice is sparse from construction.

Each Miller-loop step then calls:

```
rem, _ = f.MulPolyRings(ring, f, f, L1, L2, L3)
```

and `performDeferredRingChecks` is invoked once by the gnark backend at finalisation, running the full two-round protocol of Section ??.

6.5 Protocol and Code Flow

The following paragraphs trace each step of the two-round protocol (Section ??) to the concrete function that implements it.

The verifier is essentially the circuit which encodes all the operations into constraints which can be later verified. The prover is essentially the computation done outside the circuit via `hints`.

Prover computes Q_i, R_i (Algorithm ??)

Each `MulPolyRings` calls `polyRingMulHint`, results are provided on call, Q, R for the operation stored.

Round I: commit to $\{R_i\}$, receive challenge z

`performDeferredRingChecks` collects all R_i limbs across all groups and calls `api.Commit` to obtain z in the circuit. Internally, gnark folds all committed limbs into a global linear combination `root` and derives $z = \text{MiMC}(\text{root})$ [?].

Prover forms $Q_{\text{acc}} = \sum_i z^i Q_i$ (Algorithm ??)

z from the circuit is passed to `quotientsRLCHint`, accumulating quotients coefficient-wise in mod q arithmetic.

Round II: commit to Q_{acc} , receive challenge x

The Q_{acc} limbs are passed to a second `api.Commit` call. Gnark chains the hash: $x = \text{MiMC}(z)$, binding x to the full transcript including all R_i and Q_{acc} [?].

Verifier evaluates each polynomial at x (Algorithm ??)

`evalPolyWithChallenge` calls `InnerProductNoReduce` over the pre-computed power vector $\{x^i\}$; the result is memoised in `Poly.evaluation` to avoid re-evaluation.

Verifier checks Equation (??): $\sum_i z^i (\prod_j A_{i,j}(x) - R_i(x)) = Q_{\text{acc}}(x) \cdot P(x)$. `InnerProductNoReduce` on the per-check defects weighted by $\{z^i\}$ gives `lhs`; `MulNoReduce` of $Q_{\text{acc}}(x)$ and $P(x)$ (via `modEvalFn` or the dense fallback) gives `rhs`; `AssertIsEqual(lhs, rhs)` closes the circuit.

6.6 Implementation Details and Performance

6.6.1 Schoolbook vs Horner’s Method for Polynomial Evaluation

Horner’s method for evaluations across emulated elements causes a linear increase in the limbs count per coefficient. This necessitates reductions after each multiplication. Instead, we cache powers of the evaluation point x in $\mathcal{X} = (x^0, x^1, \dots, x^d)$ and evaluate each polynomial as an inner product of $\mathcal{X}$ and the coefficient array $\mathbf{F}$:

$$\mathcal{X} \cdot \mathbf{F} = \mathcal{X}_0 \cdot F_0 + \mathcal{X}_1 \cdot F_1 + \dots + \mathcal{X}_d \cdot F_d$$

In the emulated field context, `EVAL` in Algorithm ?? is replaced by this inner product with $\mathcal{X}$. This allows us to use the inner product optimization described below.

6.6.2 Native field operations for inner products

Modular reductions can be deferred or entirely elided for operations expressible as inner products, such as polynomial evaluations and random linear combinations. This optimization leverages the capacity headroom provided by representing small emulated limbs (e.g., 64-bit limbs) within a significantly larger native field element.

Deferred Reduction: Intermediate scalars are permitted to grow over the integers $\mathbb{Z}$ during limb-wise multiplications and linear accumulations.

Preservation of Identity: Algebraic identities are strictly preserved provided the maximum accumulated limb magnitude remains bounded below the native characteristic p .

Handling Asymmetry: In circuits with asymmetric multiplicative depths, operands may accumulate as disparate polynomial multiples of the emulated characteristic p_{emul} . Consequently, a terminal single-element reduction suffices for equivalence testing, bypassing the constraint overhead of intermediate normalization.

[Overflow safety for BN254 pairing polynomials] With 4-limb, 64-bit emulated arithmetic and $d \leq 256$, each result-limb accumulates at most $4(d+1) \leq 1028$ products of two 2^{64} -bounded values, giving a maximum limb magnitude of $1028 \times 2^{128} < 2^{138}$. For the $Q_{\text{acc}}(x) \cdot P_{12}(x)$ product, the sparse P_{12} (3 nonzero coefficients, largest $82 < 2^7$) raises this to $< 2^{146}$. Since $2^{146} \ll p \approx 2^{254}$, no intermediate limb reaches the native characteristic, so the deferred single-element reduction is safe for all polynomial evaluations in BN254 pairing verification.

7 Conclusion

7.1 Target Environments

Our approach delivers maximum impact in constraint-based environments where inversions are cheap and commitments are expensive. **Arithmetized systems** (R1CS, Plonkish/SCS) benefit because Schwartz-Zippel evaluation is efficient and field inversions cost only 2C. **Recursive proof systems** benefit most directly: the 60% commitment reduction compounds at each recursion level, since each commitment in the inner circuit becomes a constraint in the outer circuit. **ZKVMs** (RISC Zero, SP1, Jolt) benefit naturally as they express computation as constraint systems compatible with our polynomial framework. **Calldata-constrained blockchains** (Ethereum L2s, BitVM) also benefit, as fewer commitments means smaller proof transcripts.

7.2 Implementation and Performance

Table ?? presents benchmark results from our gnark implementation of the polynomial ring toolkit, measured on a Groth16 verification simulation on BN254. Our theoretical analysis (Section ??) predicted 46% constraint reduction and 75% commitment reduction per zero-bit step. The end-to-end results confirm both directions of saving: **39% SCS constraint reduction** (1,890,771 $\rightarrow$ 1,158,194) and **60% commitment reduction** (739,198 $\rightarrow$ 295,946), with strong secondary improvements across compile-time memory ($\sim$ 42–43%) and solver memory ($\sim$ 26% SCS, $\sim$ 52% R1CS).

Benchmark circuit. Both implementations were evaluated against the identical Groth16Simulation circuit², which faithfully models the pairing checks performed during Groth16 proof verification:

- **Step 1:** A 2-pair fixed- $\mathbb{G}_2$ pairing check $e(P_1, Q_1) \cdot e(P_2, Q_2) = 1$, where both $\mathbb{G}_2$ points use precomputed line evaluations (PairingCheck with fixed Q).
- **Step 2:** A single in-circuit Miller loop $ML(P_3, Q_3)$ combined with a Miller loop result f computed *outside* the circuit, followed by a final-exponentiation-equals-one assertion (AssertMillerLoopAndFinalExpIsOne).

This structure directly mirrors real Groth16 verification: the two fixed- $\mathbb{G}_2$ pairings correspond to the key-pair checks, while the combined inside/outside Miller loop models the proof-element pairing with a partially precomputed contribution.

The **gnark baseline** numbers were obtained by running BenchmarkGroth16Simulation against the upstream gnark emulated package (github.com/consensys/gnark). The **This Work** numbers were obtained from the same benchmark file compiled against our fork³, which replaces the per-multiplication deferred checks in the emulated package with the polynomial ring toolkit described in this paper. No changes to the circuit definition or test harness were required; only the underlying emulated arithmetic backend differs between the two runs. The Garaga library [?] provided an earlier proof of concept for this technique in Cairo/Starknet (2024) but lacked a systematic comparison against an industry baseline; this work provides that comparison against gnark, the standard framework for field emulation in arithmetized circuits.

²Circuit definition: <https://gist.github.com/shramee/6b02bea91e25634108338b9bce384e5e>

³Our implementation: <https://github.com/mistcash/tiny-gnark/tree/ppp>

Table 6: Performance comparison: gnark implementation, Groth16 simulation on BN254 (2 fixed-point pairings + 1 full pairing + 1 $\mathbb{F}_{p^{12}}$ mul)

Metric	Gnark Baseline	This Work	Reduction
SCS (Plonkish) Constraint System			
Nb Constraints	1,890,771	1,158,194	38.7%
Nb Commitments	739,198	295,946	60.0%
Compile Time	775.64 ms	406.71 ms	47.6%
Compile Memory	1.77 GB	1.03 GB	41.8%
Solve Time	479.61 ms	356.42 ms	25.7%
Solve Memory	510.36 MB	376.57 MB	26.2%
R1CS Constraint System			
Nb Constraints	588,013	353,881	39.8%
Nb Commitments	739,198	295,946	60.0%
Compile Time	780.40 ms	423.09 ms	45.8%
Compile Memory	2.09 GB	1.19 GB	43.1%
Solve Time	320.84 ms	214.73 ms	33.1%
Solve Memory	330.54 MB	158.51 MB	52.0%

Commitments: Fiat-Shamir `api.Commit` calls. Hardware: Apple M3 Pro (`goarch:arm64`).

Circuit benchmarked: <https://gist.github.com/shramee/6b02bea91e25634108338b9bce384e5e>

Our implementation: <https://github.com/mistcash/tiny-gnark/tree/ppp>

7.3 Final Remarks

We have shown that choosing the right granularity for polynomial identity testing—entire Miller loop steps rather than individual multiplications—yields substantial and measurable improvements. The key technique, collapsing a composite multi-operand product into a single Euclidean division check with batched quotient accumulation, is simple to implement and broadly applicable to other multi-step cryptographic computations in arithmetized environments.

Our polynomial ring toolkit is implemented in gnark’s `emulated` package and is available as a reusable, parameterised library for any emulated field and modulus polynomial. We hope this work encourages further exploration of polynomial batching granularity as a design axis for constraint-system implementations.